

Unit 1 — Lesson 2:

Constants, Variables, and Data Types

Constants and variables

Associate a name with a value

Defining a constant or variable

- Allocates storage for the value in memory

- Associate the constant name with the assigned value

Constants

Defined using the `let` keyword

```
let name = "John"
```

Defined using the `let` keyword

```
let pi = 3.14159
```

Can't assign a constant a new value

```
let name = "John"  
name = "James"
```



Cannot assign to value: 'name' is a 'let' constant

Variables

Defined using the `var` keyword

```
var age = 29
```

Can assign a new value to a variable

```
var age = 29
```

```
age = 30
```

```
let defaultScore = 100  
var playerOneScore = defaultScore  
var playerTwoScore = defaultScore
```

```
print(playerOneScore)  
print(playerTwoScore)
```

```
playerOneScore = 200  
print(playerOneScore)
```

100

100

200

Naming constants and variables

Rules

- No mathematical symbols
- No spaces
- Can't begin with a number

```
let  $\pi$  = 3.14159
```

```
let 一百 = 100
```

```
let 🎲 = 6
```

```
let mañana = "Tomorrow"
```

```
let anzahlDerBücher = 15 //numberOfBooks
```


Naming constants and variables

Best practices

1. Be clear and descriptive

✗ `n`

✓ `firstName`

2. Use camel case when multiple words in a name

✗ `firstname`

✓ `firstName`

Comments

```
// Setting pi to a rough estimate
```

```
let  $\pi$  = 3.14
```

```
/* The digits of pi are infinite,  
so instead I chose a close approximation.*/
```

```
let  $\pi$  = 3.14
```


Types

```
struct Person {  
    let firstName: String  
    let lastName: String  
  
    func sayHello() {  
        print("Hello there! My name is \(firstName) \(lastName).")  
    }  
}
```

```
struct Person {  
    let firstName: String  
    let lastName: String  
  
    func sayHello() {  
        print("Hello there! My name is \(firstName) \(lastName).")  
    }  
}
```

```
let aPerson = Person(firstName: "Jacob", lastName: "Edwards")  
let anotherPerson = Person(firstName: "Candace", lastName: "Salinas")
```

```
aPerson.sayHello()  
anotherPerson.sayHello()
```

Hello there! My name is Jacob Edwards.

Hello there! My name is Candace Salinas.

Most common types

	Symbol	Purpose	Example
Integer	<code>Int</code>	Represents whole numbers	<code>4</code>
Double	<code>Double</code>	Represents numbers requiring decimal points	<code>13.45</code>
Boolean	<code>Bool</code>	Represents true or false values	<code>true</code>
String	<code>String</code>	Represents text	<code>"Once upon a time..."</code>

Type safety

- Swift is considered a type-safe language.
- Type-safe languages encourage or require you to be clear about the types of values your code can work with
- When compiling your code, Swift performs type checking on all your constants and variables and flags any mismatched types as errors.

```
let playerName = "Julian"  
var playerScore = 1000  
var gameOver = false  
playerScore = playerName
```



Cannot assign value of type 'String' to type 'Int'

```
var wholeNumber = 30  
var numberWithDecimals = 17.5  
wholeNumber = numberWithDecimals
```



Cannot assign value of type 'Double' to type 'Int'

Type inference

```
let cityName = "San Francisco"  
let pi = 3.1415927
```

Type annotation

```
let cityName: String = "San Francisco"  
let pi: Double = 3.1415927
```

```
let number: Double = 3  
print(number)
```

3.0

Type annotation

Three common cases

1. When you create a constant or variable before assigning it a value

```
let firstName: String  
//...  
firstName = "Layne"
```

Type annotation

Three common cases

2. When you create a constant or variable that could be inferred as two or more different types

```
let middleInitial: Character = "J"  
var remainingDistance: Float = 30
```


Type annotation

Three common cases

3. When you add properties to a type definition

```
struct Car {  
    let make: String  
    let model: String  
    let year: Int  
}
```

Required values

```
var x
```



Type annotation missing in pattern

Required values

```
var x: Int
```


Required values

```
var x: Int  
print(x)
```

 Variable 'x' used before being initialized

Required values

```
var x: Int
```

```
x = 10
```

```
print(x)
```

10

Numeric literal formatting

```
var largeUglyNumber = 1000000000  
var largePrettyNumber = 1_000_000_000
```


Integer Bound Value: Just like other languages in Swift also we can find the minimum and maximum value of each integer type. Their minimum and the maximum value represent the integer bound.

Type	Size	Range
Int8	8 bit	-128 to 127
Int16	16 bit	-2^{15} to $2^{15}-1$
Int32	32bit	-2^{31} to $2^{31}-1$
Int64	64bit	-2^{63} to $2^{63}-1$
UInt8	Depend on platform	0 to 2^8
UInt16	Depend on platform	0 to 2^{16}
UInt32	Depend on platform	0 to 2^{32}
UInt64	Depend on platform	0 to 2^{64}

Size of Variables

```
var var1:Bool = false;  
var var2:String = "Hello";  
var var3:Character = "A";  
var var4:Int = 10;  
var var5:Float = 15.5;  
var var6:Double = 3.141;
```

```
// print the sizes of the variables  
print(MemoryLayout.size(ofValue: var1));  
print(MemoryLayout.size(ofValue: var2));  
print(MemoryLayout.size(ofValue: var3));  
print(MemoryLayout.size(ofValue: var4));  
print(MemoryLayout.size(ofValue: var5));  
print(MemoryLayout.size(ofValue: var5));
```

Size (in bits)

1
16
16
8
4
8

Swift Basic Input and Output

Swift Output: `print()`

Syntax of `print()`

- In the above code, the `print()` function is taking a single parameter.
- However, the actual syntax of the `print` function accepts **3** parameters

```
print(items: separator: terminator:)
```

Here,

- **items** - values inside the double quotation
- **separator** (optional) - allows us to separate multiple **items** inside `print()`.
- **terminator** (optional) - allows us to add add specific values like new line `"\n"`, tab `"\t"`
-

Swift Basic Input and Output

Swift Basic Input: `readLine()`

Example:

```
print("Enter your favorite programming language:")  
let name = readLine()
```

```
print("Your favorite programming language is \(name!).")
```

- Output:

- Enter your favorite programming language:

- Swift

- Your favorite programming language is Swift.